LANCE

Architecture, protocols and current state

Report as of 30 October 2026

LANCE project

Written by specialist strands, independently reviewed and synthesised

Last revision: 2026-10-30 15:45 (Europe/Brussels, UTC+01:00)

Documentation state as of 30 October 2026; no new experiment is reported

Abstract

LANCE is a platform for the security audit of authorised connected-object networks, developed in the context of the UMONS work of the NATO SPS G6259 programme. This report describes the pipeline architecture, the audit protocols and their evidence contracts, the experimentation harnesses and their hermetic sandbox, the execution infrastructure and the delivery pipeline, and the actual state of the benchmark. It succeeds the 15 October 2026 overview, which skimmed over the architecture and the protocols : those parts are developed here down to the code contract.

The document consistently distinguishes what is implemented, what is measured and what remains proposed. No new experiment is reported : the measurements cited are those already recorded, with their date. The limits of each component are explicit, including those of the sandbox, whose guard is not an exhaustive native-code sandbox.

Contents

1 Executive summary **2**

2 System overview **4**

3 Scope and method **5**

4 End-to-end pipeline architecture **5**

5 Audit protocols and evidence contracts **8**

6 Experimentation harness and hermetic sandbox **11**

7 Execution infrastructure and delivery pipeline **13**

8 Benchmark and scientific progress **16**

9 Conclusion **20**

1 Executive summary

This page answers four questions without jargon : what the system does, where it stands, what is proven, and what has to be decided.

1.1 What the system does

LANCE conducts a security audit on a network of connected objects *whose ownership and authorisation are established*. It explores the network, issues hypotheses of flaws, attempts to verify them by observable evidence, and then writes a report. Two policies can carry out this audit : a rule-based automation and an agent based on a language model. The project seeks to determine which of the two succeeds better, with identical tools, evidence and budgets.

1.2 Where the project stands

Table 1 – Status in one line per strand.

Strand	Status	Detail
Audit engine	mature	Six phases in service, four modes, execution boundary that logs every call
Evidence	mature	Briefing/oracle/evaluator contract applied to five families ; the log is authoritative, not the decision
Harness	consolidated	Core shared since 29 October ; sandbox executable outside the laboratory
Infrastructure	restored	Three instances, automatic delivery pipeline, seven defects fixed
Benchmark	partial	One hypothesis measured out of six, one family rejected, no inference computable
Scientific proof	absent	No superiority of the model has been established to date

1.3 What is proven, and what is not

Proven. Every tool call is traced and bounded ; a lost piece of evidence interrupts the run. A harness can run outside the laboratory, under a guard whose blind spots are declared. On the best-documented family, a third script, which reads by convention and uses no model, obtains the same result as the agent, which *absorbs* the gain attributed to the model. The corpus is mostly detectable by signature, which makes a null gap expected by construction.

Not proven. That a model does better than a competent script. That a gap is significant. That the results generalise beyond eight cases and one property.

1.4 The three decisions that govern what follows

1. **The independent families.** The inference tool was repaired on 30 October 2026 and now refuses to produce a number it cannot defend. What remains is the families themselves : the corpus holds at most three defensible ones, and a decision needs five. Either build them, applying the filter that one hour of observation established, or publish a bounded methodological result and declare it as such.
2. **The opening condition.** Several families remain undecidable for lack of a written reference automation. Either write it, or explicitly renounce these families.

1.5 How to read this report

Part 4 describes the path of an audit. Part 5 describes what makes a result verifiable. Part 6 describes the experimental tooling. Part 7 describes the machine and the delivery. Part 8 establishes what is measured. The conclusion confronts these strands and notes their contradictions.

Each statement refers to the file that supports it. Behaviours that the code does not allow to be established are flagged as such rather than filled in.

2 System overview

Before going into the contracts, a map of the system and of the laboratory. It situates the components described below and shows where evidence passes.

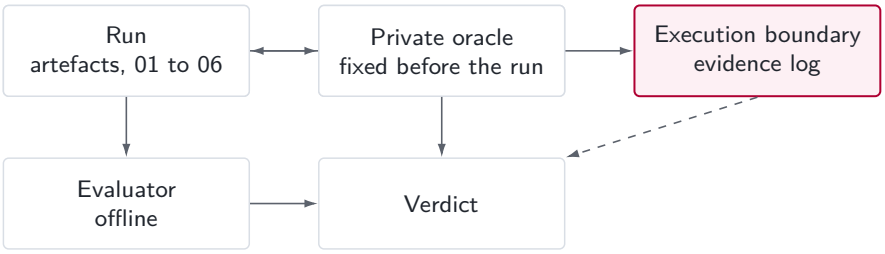


Figure 1 – The journey of a piece of evidence. The tools produce logged observations ; the private oracle, written before execution, makes it possible to recompute the verdict from the log without believing the system's decision.

Three properties can be read from this map.

The log is the source. A result is not the conclusion written by the policy, but what the evaluator recomputes from the archived observations. A cited piece of evidence that does not appear in the log is rejected.

The oracle precedes the run. It is fixed before the policy runs, and a finding that contradicts it invalidates the trial instead of correcting it. This precedence is what prevents the answer from being adjusted after the fact.

The sandbox is not the measurement. It executes the deterministic arms without a model or a laboratory ; it instruments the contrast, it does not measure it.



Figure 2 – The delivery pipeline. No execution work takes place on the development workstation : the push triggers validation, which deploys to the container.

3 Scope and method

This report answers a reading question : *how is the system built, and where does it actually stand ?* It does not replace the reference pages of `docs/`, which describe the current behaviour, nor the detailed protocols of `research/`, which carry the reasoning.

Each part was written by a specialist working on the `dev/1` branch, then reviewed against that same code by an independent reader. A cross-cutting synthesis was produced separately. Statements that cannot be verified in the repository are flagged as such rather than filled in.

The report is dated : it describes a state, not a contract. Later behaviour may contradict it without making it false.

4 End-to-end pipeline architecture

This part describes the effective path of a LANCE audit, from the selection of an execution mode to the files deposited in the run folder, relying exclusively on the code of the `dev/1` branch. The statements refer to the module and, where useful, to the function that implements them ; behaviours that the code does not allow to be established are flagged as such rather than filled in by inference.

4.1 Overview and mode selection

The application does not carry the audit logic : it validates the request, guarantees a single active run per instance, streams the SSE events and re-reads the artefacts (`docs/architecture/run-modes.md`). It is `make\_runner` (`src/agent/run\_modes.py`) that chooses the execution strategy. For `lance` and `rules`, it instantiates a `Pipeline` and wraps it in `LanceRunner`, an adapter that reuses the pipeline object and its public API. For `vanilla` and `scripted`, it builds a `SimpleRunner`, a standalone class that shares with the pipeline only the tool execution boundary, the evidence log, the cost tracking and the laboratory reservation.

Table 2 – Execution modes and corresponding boundary.

Mode	Engine	Model	Effects
`lance`	`Pipeline` (6 phases)	required	scenario, deployment, evaluation
`rules`	`Pipeline` (phases 3–4)	none	scenario and deterministic audit
`vanilla`	`SimpleRunner`	required	bounded Nmap probe on CIDR
`scripted`	`SimpleRunner`	none	versioned procedures

The `rules` mode is not a second engine : `make\_runner` sets `decision\_policy="rules"`, `phases=[3,4]` and `execution\_profile="full"`, and rejects any provider, direct network or monetary budget. The `vanilla` and `scripted` modes, conversely, reject `scenario\_id` and any harness-specific setting. The qualifier `full`/`compact` designates the orchestration of the pipeline, never the type of runner.

4.2 The six phases and their deliverables

The declarative registry `src/agent/registry.py` (`AGENTS`) assigns to each phase a prompt template, a tool group, a validator and a deliverable name. The routing is explicit in `src/agent/phases/registry.py` : `PHASE\_MODULES` maps the numbers 1 to 6 to fixed modules, without dynamic import.

Table 3 – Phases, modules, main deliverables and associated artefacts.

Phase	Module	Main deliverable	Associated artefacts
1	graph/run.py	01_graph_analysis.md	01_graph_evidence.json
2	recon/run.py	02_recon.md	02_recon_evidence.json, 02_topology_edges.json
3	analysis/run.py	03_vuln_analysis.json	03_vuln_analysis_raw.json, 03_phase3_status.json, 03_device_*
4	verification/run.py	04_exploitation.json	execution_errors diagnostics
5	intrusion/run.py	05_intrusion.json	05_intrusion_context.json
6	report/run.py	06_report.md	06_report_analysis.md, per-section sheets

Phase 1 projects the results of the graph tools into a log of facts independent of the model's discourse (GraphPhase._build_graph_evidence_projection), and the runner rejects a submission whose per-device coverage is incomplete (src/agent/core/runner.py). Phase 2 lets the model choose the probes, but 02_recon.md is rendered from the tool log, not copied by the model. Phase 3 runs in three steps : scan.scan_phase (deterministic observations), devices.analyze_devices (analysts per device) and then run.aggregate_phase, which normalises, deduplicates and projects the findings and writes 03_vuln_analysis.json. Phase 4 launches micro-agents per vulnerability and aggregates their results. Phase 5 produces its assessment from the log : the model supplies only an end marker, and the status remains objectives_status=not_certified. Phase 6 is particular : its AgentConfig declares no tool and max_turns=1 ; the report is assembled deterministically from sheets, the model writing only a bounded analyst note (06_report_analysis.md).

4.3 Graph and run context

The graph is built in two layers. src/loader.py reads the infrastructure YAML into dataclasses (load_yaml) and then populates a backend (build_graph, default backend NetworkXBackend). src/graph_backend.py defines the abstract interface GraphBackend (get_device, get_neighbors, find_all_paths, get_attack_surface, get_graph_stats, to_dict) and its NetworkX implementation on a DiGraph. Two details deserve to be stressed : get_neighbors returns the sorted union of successors and predecessors, and find_all_paths works on an undirected view (to_undirected) in order to find paths in both directions, including through the abstract backend.

The state belongs to the run. Each Pipeline creates an immutable RunContext (run folder and GraphContext) ; GraphContext carries the backend, the infrastructure, the CVE reports, the risk scores, the scenario or discovery topology, the weighted graph and the latest delta. The graph tools retrieve this context via a ContextVar (src/agent/core/graph_context.py), without modifying the arguments exposed to the model ; graph_bound and bind_graph_callable explicitly capture the owner, including for thread workers, rather than relying on implicit context propagation.

4.4 The tool governance boundary

All profiles and all modes go through wrap_tool (src/agent/core/executor.py). On each call, the wrapped function : (1) checks that the previous piece of evidence could be archived, cooperative stop and the budgets via check_execution_limits ; (2) atomically increments a counter and refuses the call if max_tool_calls is reached ; (3) assigns a reference tc-<uuid> ; (4) applies the

scope refusals ; (5) executes the tool ; (6) systematically archives an entry in `tool\_calls.jsonl`, including for a refusal or an exception.

Table 4 – Controls applied by `check\_execution\_limits` and `wrap\_tool`.

Control	Mechanism	Consequence of a failure
Evidence integrity	flag `\_evidence\_integrity\_failed`	`EvidenceWriteError`, run not verifiable
User stop	`\_stop\_event` (experimental scope)	`RunStopped`
Time budget	`max\_duration\_s` via `time.monotonic()`	`BudgetExceeded`
Monetary budget	`tracker.check\_budget()`	`BudgetExceeded`
Call budget	counter under `\_artifact\_log\_lock`	`BudgetExceeded`
Scope	`\_intrusion\_scope\_violation`	refused call archived as JSON
Free execution	`python\_exec`	refusal `unverifiable\_execution\_scope`

Two properties are important for the interpretation of the results. On the one hand, a failure to write the log is never ignored : it is converted into `EvidenceWriteError`, which interrupts the run rather than leaving untraced actions. On the other hand, the cost limit of `check\_budget` applies to observed usage (`total\_cost()`), not to a reservation of in-flight calls ; the code docstring says so explicitly. The scope is checked on the destination fields (`ip`, `host`, `target`, `url`) against the `target\_subnet` of the run ; an action without a verifiable destination is refused upstream (`python\_exec`), and the module header specifies that these are application guards, not a system sandbox.

4.5 Artefacts and cost tracking

The run folder is created exclusively under `output/agent/` (`Pipeline.\_\_init\_\_`) ; in the event of a collision, a hexadecimal suffix is appended and no earlier folder is overwritten. `run\_meta.json` records phase, profile, policies, commit and limits ; `cost\_summary.json` is written in the `finally` block of the pipeline, and therefore even on exception. The collection contract (`src/benchmark/artifacts.py`) defines a positive list of allowed files (`\_ALLOWED\_ROOT\_FILES`) and `03\_device\_*.json` patterns ; `create\_submission\_bundle` produces a zip equipped with a SHA-256 manifest and re-checks the manifest after writing. On the agent side, a deliverable attempt is archived in `.attempts/` and recorded in `deliverable\_attempts.jsonl` ; the route `GET/api/runs/\{id\}/download/zip` excludes private files and hidden entries.

Cost tracking (`src/agent/cost\_tracker.py`) resolves the price once per phase (`\_resolve\_pricing`) and records it in `PhaseUsage`, so that a completed run cannot be re-evaluated after the fact. Each phase carries the source of the price (`dynamic\_catalog`, `static\_catalog`, `subscription`, `default\_estimate`) and a `cost\_is\_estimate` indicator. `summary()` explicitly distinguishes the cumulative duration of the agents (`total\_agent\_duration\_s`) from the wall-clock time (`wall\_clock\_duration\_s`) ; the former does not measure the time actually elapsed.

4.6 What the architecture does not guarantee

The architecture is isolated per run for the deliverables and the graph, but it is not entirely so : `src/agent/phases/README.md` indicates that the phase services may still call the common engine and that the simultaneity of scenarios on a shared laboratory is not guaranteed. `reserve\_lab` is only active if `LANCE\_LAB\_LOCK` is set and is only reentrant in the same thread. Moreover, without

an explicit parameter, the interactive pipeline imposes neither a call ceiling nor an analysis timeout per device : the budgets of table 4 are therefore conditional.

Three points could not be established by reading the code. *One* : no mechanism for archiving or compacting run folders was found in `src/` (no occurrence of `tarfile`, `shutil.make\_archive` or equivalent) ; retention appears to be that of the file system, which the documentation does not describe. *Two* : `02\_recon\_evidence.json` is read by `src/agent/scanner.py` and `src/agent/core/runner.py`, but its main producer has not been identified ; asserting it would be a supposition. *Three* : `provider\_events.jsonl` is recorded by the provider callback (`core/runner.py`) via `provider\_diagnostics`, and the exact content of the fields has not been verified line by line. Finally, a `completed` status means that the runner has finished its work, never that the target is exhaustively covered nor that an access has been proven (`docs/architecture/run-modes.md`).

5 Audit protocols and evidence contracts

This part describes the experimental infrastructure : the audit families, the three-part contract that structures them, the private oracle, the offline evaluator, the V0 fidelity control, the H2 family and the confirmation metric. Each statement refers to a file in the repository. The reading distinguishes three registers : *implemented* (the code exists and enforces the property), *measured* (a documented run yields a value) and *proposed* (declared in a protocol, without code).

5.1 Implemented families

The repository contains five families, each with a centralised contract file, a deterministic arm, a bounded LLM arm and a launcher reserved for the laboratory.

Table 5 – Families, what they measure and what they leave out of scope.

Family	Contract	What it measures	What it does not measure
SSH	`ssh\_reference.py`	Whether a dictionary credential reads the witness object that the legitimate control also reads	Fixture reset, provenance of the model
MQTT	`mqtt\_reference.py`	Whether an anonymous subscriber receives the fresh private witness ; in a healthy posture, the evaluated refusal	Purge of retained messages, broker reset
API	`api\_reference.py`	Coverage of the four authorisation boundaries	The baseline limit is *declared* : the script reads no collection
V0	`v0\_control.py`	Fidelity of execution : a public procedure executed and proved	An assumption about the model
H2	`mqtt\_formats.py`	Reading of a frozen MQTT presentation	Network collection, field semantics, statistical independence

The limit is not merely editorial : it is recorded in each result, under the key `scope`, and in `measurement\_status:evidence\_only\_not\_admitted`.

5.2 A three-part contract

Each family separates three artefacts, and the separation is enforced by the code.

The public briefing contains only the task. `ssh\_reference.validate\_briefing` requires exactly the expected set of fields, which forbids adding a posture or a verdict. The SSH dictionary cannot contain the control identity, and `mqtt\_reference.validate\_briefing` refuses a public subject equal to the private subject. The posture is never written in the briefing : it is inferred from the oracle.

The private oracle is bound to the briefing by the latter's digest, and its fields are bounded by a closed set. It states which state was installed (F0 healthy or F1 vulnerable) and which fact is observable ; it never states which decision the policy should return.

The offline evaluator rereads the log and the terminal decision, then recomputes the outcome. The evidence file is `tool\_calls.jsonl` for SSH, MQTT, API and V0 ; it is `journal.jsonl` for H2, reattached by `Journal.replayed`, which refuses a record originating from another dossier. Each record carries a reference produced by the shared executor (`src/agent/core/executor.py`).

5.3 The oracle : construction and guarantees

The oracles are produced by `oracle\_for` functions, present in three modules. SSH builds its own inline, from `case\_configuration`, which draws the object with `secrets.token\_hex(32)` ; MQTT produces two fresh witnesses per trial ; the API randomises organisations, users, objects and collections at each trial.

Three barriers prevent deriving the answer from the policy. The first is structural : the evaluator receives only the briefing, the oracle, the journal and the decision, and the oracle is written before the arm runs. The second is cross-validation : a briefing that would name an oracle file fails, and an unbound oracle fails. The third is the contradiction verified over the entire journal : under a protected oracle, an observed disclosure makes the trial `oracle\_conflict` rather than a success. The oracle therefore cannot be adjusted after the fact.

The witness (`canary` in the code) is a fresh value drawn at each fixture start, whose digest is in the oracle and whose value appears nowhere in the briefing. It proves freshness and qualifies the control : the legitimate control must have read the value. Without a witness, an observed refusal might be mere silence, which MQTT explicitly refuses.

5.4 The offline evaluator : recount and statuses

The terminal decision is never taken on trust alone. The evaluator requires that its evidence references be a subset of the journal, and otherwise returns `outcome:unknown`. It then recomputes per observation type : an archived request must be identical to the requested request, and the payload is extracted only if the return code and the framing are consistent. Unbound evidence does not count, even when cited.

In F1, success requires a finding, a proven weak access and a functional control. In F0, it requires the absence of a finding, a functional control and the explicit refusal of each pair of the dictionary. The code distinguishes *non-success* (trial carried to completion, contract not satisfied) from *missing* (evidence or decision absent). The distinction is not cosmetic : a missing item does not enter a success-rate denominator, a non-success fills it. In V0, `instrument\_defect` is added when the script prevails and the LLM fails : this is a defect of the instrument, not a weakness of the model.

5.5 The V0 control

V0 is the validity gate, not a hypothesis. The task is solvable without inference : every subject, identifier and window appears in the briefing.

A *saturating control* is a control that both arms pass everywhere : it detects no defect. The manifest `benchmarks/experiments/v0-control-manifest.json` freezes `X=0.9`, 24 steps and 3 repetitions, and declares `saturating:true` with its reading : the control cannot detect a defect rarer than one failure in three. The evidence is a run with three repetitions in which both arms succeed everywhere. The threshold is read by `frozen\_threshold`, which refuses a manifest that is not frozen or that was calibrated on a different number of steps.

5.6 H2 : three arms, one absorbed prediction

H2 tests whether a model reads a presentation that no declared parser covers. The eight dossiers of the corpus cross `\{reference,variant\}` and `\{raw,normalized\}`, two dossiers per group. Three arms share the same journal and the same verdict vocabulary.

Table 6 – Success rate per group for the three H2 arms (measurement of 22 October).

Group	Declared parser	By convention	LLM
`reference/raw`	1.0	1.0	1.0
`reference/normalized`	1.0	1.0	1.0
`variant/raw`	0.0	**1.0**	**0.5**
`variant/normalized`	1.0	1.0	1.0

The LLM gain holds for one dossier in two. But the conventional arm covers both without a model, since the convention suffices to read the variant. The control therefore publishes `convention\_gain\_absorbed:true` : what the LLM gains over the declared parser, a script that tries also gains. The prediction remains literally upheld, but it no longer distinguishes the model from the declaration.

5.7 The strict-v3 contract and confirmed metrics

The contract `src/benchmark/strict\_v3.py` contains no scoring code : it normalises the contract per vulnerability and derives a conservative value for earlier ground truths. The `confirmed.*` metric is built in the comparator by prefixing the funnel stage names (`src/benchmark/funnel.py`) : `candidates`, `filtered`, `confirmed`. The `confirmed` stage denotes the deduplicated findings whose evidence has been independently validated against tool outputs. The code states it : a confirmation whose evidence is invalid is not a true positive. The stage is unavailable if the exploitation file or the provenance journal is missing ; it is never reconstructed from the survivors.

5.8 What these contracts do not prove

1. **Nothing about the ground truth.** The oracle is supplied by the operator ; each family records this in its scope. A false oracle makes all verdicts false.
2. **Nothing about admission.** Campaign admission requires the explicit adoption of the strict-v3 metrics, two independent families, three repetitions per posture and an attested resource parity : conditions not met by the calibrations.
3. **Nothing about arm parity.** SSH is not an identical task : on the LLM side, the model reads the result and judges equality with the control itself, while the deterministic arm compares the payloads ; the evaluator recomputes off-band. The budget-parity condition covers only declared budgets, not the hidden calls of the script arm.

4. **Nothing about a rare defect.** V0 is saturating : with three repetitions, it detects only a repeated defect.
5. **Nothing about the real space.** H2 holds for eight hand-written dossiers, a single variant schema and a single property.
6. **Nothing about semantics or injection** : these are H4 and H6, outside the scope of these contracts.
7. **Nothing about comparative cost** : the deterministic arm consumes no tokens.
8. **Certain guarantees belong to the written protocol**, not to a code assertion : the arm count of H2 and the invariant that an observation cannot answer two steps.

6 Experimentation harness and hermetic sandbox

This part describes the recent work that unified the life cycle of calibration harnesses and added a hermetic sandbox executable outside the laboratory. It separates three objects : the anatomy of a harness, the shared core that builds its execution object, and the sandbox that executes a deterministic arm under an audit guard. Three commits trace it on the `dev/1` branch : `11b820c` (sandbox), `02320d5` (closure of the guard's gaps) and `5101523` (shared core).

6.1 Anatomy of a harness

A harness is a set of files with a single responsibility, under `benchmarks/experiments/`. The MQTT harness gives its complete form. The **contract** is `mqtt\_reference.py` : it validates the briefing, binds the tool to the execution boundary, provides the **D1 adapter** through the deterministic arm `run\_rules` and recounts the run through `evaluate\_trial`. The **LLM arm** is `mqtt\_reference\_llm.py` ; `mqtt\_formats\_llm.py` is its H2 variant, which receives no parser. The **live controller** is `mqtt\_reference\_live.py`, not to be confused with the arm : it builds the LLM agent, the wrapper of a `RecordingClient` that journals every transport call, and writes `decision.json`, `cost\_summary.json` and `run\_meta.json`. The **deployment** is `mqtt\_reference\_proxmox.py` : the class `Proxmox` submits operations via `ansible-playbook`, the class `Fixture` starts the broker from the allocation.

The same anatomy is repeated for SSH, the API and the V0 control, with one important difference : the H2 harness (`mqtt\_formats.py`) has neither a Proxmox deployment nor an active live client in the sandbox, since its corpus is frozen.

6.2 The shared core

The core exists for a precise reason, recorded in its header : the governance boundary of `src/agent/core/executor.py` reads a fixed set of attributes via `getattr` with a default value. An omission therefore manifests only when a tool call misbehaves. Each harness rebuilt this object by hand ; the core replaces the copies with a single `build\_run`.

`REQUIRED\_ATTRIBUTES` enumerates thirteen attributes : the run directory, the three budgets, the archival lock, the evidence lock, the call counter, the start clock, the limit reason, then the scope, the context, the policy and the cost tracking. One attribute deserves a particular comment : `\_evidence\_integrity\_failed`. `check\_execution\_limits` reads it as its first instruction and raises `EvidenceWriteError` if it is true ; `wrap\_tool.archive` sets it when the archive cannot be written. This lock documents a rule : lost evidence must remain visible, and the run becomes unverifiable. A harness that forgot it would fall back to the default value and pursue a run whose archive is holed.

The refusal is two-level : `build\_run` checks the missing attribute and the absent context key before returning the object, and `require\_attributes` replays the same check on an arbitrary object. A deterministic variant declares neither model nor provider.

Coverage remains incomplete : the four *_live.py harnesses go through a local function of the same name new_run, and the core's new_run has no direct caller ; the delegation goes through build_run.

6.3 The sandbox

benchmarks/experiments/sandbox.py proves, on a development workstation as well as in continuous integration, that a harness executes without a broker, without a model and without a laboratory. It loads the frozen corpus (eight fixtures) and, for each dossier : validates the dossier, opens a journal, executes the chosen arm, writes the artefacts, then recounts via the existing evaluator. Five artefacts are produced per trial : dossier.json, oracle.json, answer.json, journal.jsonl and evaluation.json. A sandbox-report.json aggregates the trials and the guard block.

The arm is an explicit parameter, and an unknown name raises an error. This choice is a correction : the original version read the arm from the corpus, which does not declare it, rendering the conventional arm unreachable and causing the report to invent the tested policy.

6.4 The audit guard and its limits

The guard relies on sys.addaudithook. The Guard class remains installed in the process, but inspects events only when armed : the active() context manager bounds the enforcement to the run. install_guard installs the hook only once and then rebinds the authorised directory and the list of violations at each run ; without this rebind, a second run would observe the first run's directory and stifle its own violations. The blocked events cover sockets, subprocesses, the exec/spawn/posix_spawn family, ctypes and rename and delete operations. Writes are handled separately : the open event is refused if the flags contain a write mode and if the resolved path leaves the authorised directory.

The honesty of this mechanism can be read in commit 02320d5. A read-only audit broke the first version with two measured escapes : os.posix_spawn started a process, and ctypes.CDLL(None).system() executed native code, while the report announced a hermetic execution without violation. The report was therefore corrected on three points : the key hermetic is replaced by guard_clean, which states only the absence of a listed event ; a guard block declares the mechanism, the blocked events and, explicitly, the blind spots ; and claims_isolation is set to false. This last indicator is false by construction : an audit hook observes events of the Python interpreter, not system calls. It does not see native code reached outside ctypes, nor a write through a descriptor opened before arming, nor a system call issued outside CPython.

This limit has a methodological consequence : guard_clean:true means "none of the listed events occurred", never "the run is isolated".

Table 7 – What the guard covers, and what it leaves unobserved.

Covered, with refusal raised	Unobserved
Sockets : connection, resolution, sending, listening	Native code reached without ctypes, Popen or posix_spawn
Processes : subprocess, os.system, exec*, fork, spawn*, posix_spawn*	Writes through a descriptor opened before arming
ctypes : loading, symbol resolution, call	System calls issued outside the CPython interpreter
Rename and delete : os.rename, remove, replace, unlink, shutil	Any behaviour after disarming : the hook remains installed but inert
Writes via open outside the authorised directory (resolved paths)	No guarantee of process isolation

6.5 What the sandbox demonstrates

On the eight frozen fixtures, the declared-parser arm succeeds in six trials out of eight : the two failures are the `variant/raw` trials, where the declared parser does not cover the schema and abstains on every message, hence an erroneous conclusion. The conventional arm, which reads the keys without a declared schema, succeeds eight out of eight. Success is not counted on the conclusion alone : the evaluator imposes it as a correct conclusion, without contradiction and without any erroneous verdict.

Both arms produce the same offline lesson as H2 : the failure on `variant/raw` does not stem from a lack of intelligence but from a boundary of the declared parser, and a script that tries restores the coverage. The sandbox instruments the contrast ; it does not measure it on a model. No LLM arm appears there, no pre-flight attestation is produced, and neither `pair\_admissible` nor `mt\_admissible` emerges from it.

6.6 Sandbox limits

Five limits must be read against the preceding results. First, the sandbox involves no model : its success is a property of the harness, not an experimental result, and the report carries `claims\_measurement:false`. Next, `guard\_clean` is a Python observation and not a proof of isolation. Third, the scope is partial : the negative tests that prove the hook really refuses network, processes, `posix\_spawn` and `ctypes` go through direct invocation of the guard, and no end-to-end test confirms that the run actually triggers the installed hook ; the detection path is proved only at unit level. Fourth, the corpus is frozen, has a single scenario and eight fixtures, which suffices to instrument the contrast but not to estimate a rate. Finally, the sandbox bypasses the governance boundary it is meant to accompany : it calls the arm directly, without building a run or passing through the tool wrapper, and the journal is written by the harness itself ; the shared core and the guard remain two separate instruments, each verified in isolation.

6.7 Tests

The set counts twenty-four tests : fourteen for the sandbox and ten for the core. They prove four properties. For the core, the run exposes the governance surface, an incomplete run is refused at construction, `check\_execution\_limits` accepts a new run, and the exceeding of the call budget is raised by the tool wrapper and not by the limit check. For the sandbox, the eight fixtures produce the five artefacts and a consistent report, and the arm is declared and never read from the corpus. For the H2 lesson, the declared parser fails on at least one trial when the conventional arm succeeds everywhere. For the guard, negative tests refuse writes outside the directory, the network, subprocesses, `posix\_spawn` and `ctypes`, while a test verifies that the hook remains inert after disarming and that a second guard observes its own directory.

What these tests do not prove : none executes an LLM arm, none measures a confidence interval, and none verifies that the installed hook is reached during the run rather than only when it is called by hand.

7 Execution infrastructure and delivery pipeline

7.1 Laboratory topology

The execution of LANCE relies on a single Proxmox host and one LXC container that carries the entire platform. The playbook `benchmarks/ansible/playbooks/deploy\_master.yml` creates, on the Proxmox host group, an unprivileged LXC container with identifier 200 named `nato-master` (Debian 13, 20 GB, 4096 MB of memory, 2 cores, `nesting=1`, `onboot1`). Two interfaces are attached :

eth0 on vmbr0 in DHCP for management, and eth1 on vmbr1 in 192.168.100.200/24 for the scenario test network. The playbook adds the TUN device authorisation and the mount /dev/net/tun, required by Tailscale, then rebuilds an OUTPUT chain of iptables : on eth0 only the local interface, the ESTABLISHED,RELATED states, DNS, HTTP, HTTPS, SSH, the Proxmox API (port 8006), UDP 41641 and the apt cache are allowed ; everything else is rejected. On eth1, outbound traffic is fully allowed. eth1 has no gateway, which confines the scans to the isolated network.

The Proxmox host name is given as pve-nato by AGENTS.md, objectifs/ETAT-COURANT.md and docs/guides/umons-llm.md, but neither the playbook nor docs/guides/development-instances.md names it : the versioned Ansible inventory is a template whose address literally equals <PROXMOX_IP>, and the only constant readable in the repository is the container identifier, 200. The actual configuration lives outside the repository.

7.2 The three instances

The script scripts/deployment/deploy-instance.sh fixes the slot, port, directory and service, and rejects any other name. The stable service is produced by the playbook ; the development units are written by the script.

Table 8 – Instances deployed on the master container.

Slot	Branch	Port	Checkout and service
main	main	8501	/opt/nato-smartcity-iot / nato-fastapi
dev-1	dev/1	8502	/opt/lance-dev-1 / lance-dev-1
dev-2	dev/2	8503	/opt/lance-dev-2 / lance-dev-2

Each slot has its own Python environment, its database, its .env file and its artefacts. The shared configuration is written into /var/lib/lance/<slot>.env and imported by a systemd override ; these entries feed GET/api/environment, which returns the slot, the branch, the commit and the activation of the lock. After start-up the script checks that the HTTP response announces exactly the validated commit and the expected slot, and that the lock is active.

The common lock is a single file, /var/lib/lance/lab.lock, opened by exec9>> and then locked by flock-x9 before any mutation ; the comment is explicit : never delete this file. The same value is passed to the APIs, which allows the three instances to coordinate. Waiting is cooperative, without arrival ordering, and kept in memory. A deployment of a development slot checks that the stable instance announces the lock : an older stable instance would not participate in it.

7.3 The runner and the delivery pipeline

The GitHub Actions runner is installed in /opt/actions-runner, registered with the label nato-master and run as a systemd service.

The pipeline reads as four links. A push to main, dev/1 or dev/2, or a merge request into these branches, triggers .github/workflows/benchmark-integrity.yml. The first job runs .github/scripts/ci_scope.py, which classifies the modified files and publishes the full flag. The second builds the image of the isolated worker and checks the absence of ground truth. The deployment jobs then call .github/workflows/update-master.yml, which runs the deployment script with the slot, the validated SHA, the branch and the remote repository.

Table 9 – Triggers and paths through the pipeline.

Event	Path	Deployment
documentation `push`	classification to `full=false` : diff of the previous commit and whitespace check	none
code `push`	full tests, scenario and Ansible checks, sandbox, worker image	`main` to 8501, `dev/*` to 8502 or 8503
Merge request	same path according to classification	none
Manual trigger	`full=true` forced	chosen slot, or checks only
Version tag	`full=true` forced	`main` only

The light path covers the Markdown, LaTeX, PDF and image files under `docs/`, `objectifs/` and `research/`, as well as a closed list of presentation files. The classification keeps deletions and the two paths of a rename ; any undetermined event falls back to the full path, and a failure blocks the deployment. A documentation push therefore restarts no instance.

On the machine side, the update script fetches the SHA and applies a fast-forward merge for `main`, or a detachment after checking for a clean tree for the development branches ; it then verifies that `HEAD` equals the validated SHA. Before that, the service is stopped, the lock acquired, and the instance queried in order to refuse mutating active or pending work. In the event of a failure after stopping, the service remains stopped : no automatic rollback to an earlier version is performed.

7.4 Corrections to the deployment script

The history of the script shows a tight sequence of commits on 29 October 2026. The founding version checked the host identity by a hard-coded address and required the root user.

Table 10 – Successive corrections and cause addressed.

Commit	Correction	Cause addressed
`6b0a9ff`	address in an environment variable, `sudo` re-execution, checks of the stable instance and of the configuration made optional, Codex CLI under `/usr/local/bin`	a rebuilt container broke every deployment : identity, privileges and neighbouring instances assumed inherited
`433e2a3`	identity deduced from the `tailscale0` interface, failing a variable	removes the last constant dependent on a rebuild
`a568c68` to `dcd5903`	absolute path, execution via `bash`, propagation of the runner name and of the whole deployment environment	`sudo` does not resolve a relative path nor preserve the environment
`150eb8a`	Codex CLI download failure tolerated	under `set-e`, the failure stopped the deployment and left the instance stopped

Each correction corresponds to a concrete failure mode, not to an aesthetic refinement. The same identity logic now applies to the benchmark controllers, with an explicit suspension variable, and a dedicated test covers it.

7.5 The UMONS model and its relay

The `ollama-umons` provider is reached through a local SSH transfer on the Proxmox host, towards the university cluster (`docs/guides/umons-llm.md`). The path was identified and restored on 4 October 2026, with a successful minimal generation ; the authentication observed is interactive. The restoration command is documented, as are the listening checks. The guide is explicit : neither start-up at boot, nor reconnection after the process ends ; the administration of the downstream GPU server remains to be confirmed with the administrator. The fragility is established in the facts : the relay went down again on 22 October, and it was the provider pre-flight that named the failure without executing any arm.

7.6 The outage of 29 October

The outages actually traced in the repository for that day are those of the container rebuild, which rendered obsolete the fixed address and the assumption of a privileged runner. By contrast, *no mention of a multipath default route on the laboratory router exists in the repository* : the search on the corresponding terms returns nothing. This fact, observed in session but not recorded, must not be presented as documented. It constitutes a gap in the project's memory, since the same outage will recur at the next router restart.

7.7 Known fragilities

— **Unreplicated failure points** : single Proxmox host, single container, single runner. The runner registration tolerating failure, an absent runner does not block the playbook but blocks every deployment.
— **UMONS relay** : no supervised service, interactive authentication, address still fixed in the examples of the guide although commit `e54da24` claimed to have stopped fixing addresses in the documentation. Restarting LANCE does not repair the relay.
— **Cooperative lock** : without arrival ordering, in memory, not covered for an Ansible script launched directly, an old checkout or a manual intervention. This is not security isolation between collaborators.
— **Manual recovery** : a failure after stopping leaves the service stopped, with no automatic rollback.
— **Host identity** : without a declared variable, the identity depends on the presence of any address on the Tailscale interface ; the script does not verify that this host is indeed the one that should host the instance.
— **Partial documentation** : the name of the Proxmox host and the physical segment correspond only imperfectly to the playbook ; the versioned inventory is a template.

8 Benchmark and scientific progress

The benchmark is not a list of scenarios but a register of hypotheses. This part describes what is actually measured, what blocks inference and what can be published today. Every figure is recomputed from an artefact of the repository, or reported as read in a document.

8.1 The hypothesis register and the V0 gate

The register places before every hypothesis a *validity gate* V0 : if a control task solvable without inference fails on one of the two arms, no downstream result is readable. The threshold is frozen in `benchmarks/experiments/v0-control-manifest.json`, which carries a value of 0.9, 24 steps, 3 repetitions and the entry `saturating:true`. The calibration pass returns a valid control on both

arms, three times out of three. The register draws from it a *partially passed* gate : the instrument is readable for execution parity and for a repeated defect, not for a rare defect.

Two caveats must be stated. On the one hand, V0 covers only MQTT : the SSH harness keeps a fixed credential dictionary and is therefore not an instance held apart. On the other hand, and this is a methodological flaw, the threshold is not applied by the gate : the gate function reads the four outcomes on "all trials succeeded", whereas the threshold check is computed separately and conditions nothing. With three repetitions, two successes out of three equal 0.67, so the two formulations coincide, but by numerical coincidence, not by application of the threshold.

Table 11 – Status of the register, cross-checked with the protocols.

Item	Status	What this means
V0	partially passed	Valid control on MQTT, threshold frozen but not applied by the gate ; saturating control ; SSH not covered
H1	conditional, not opened	The identity-chaining family does not exist in the coverage plan
H2	measured on 22 October	Prediction held, gain absorbed by an arm by convention
H3	not implemented	The public-contract families are planned
H4	not opened	The vendor-register family does not exist in the corpus
H5	conditional, planned	Budget counter-test to be written
H6	planned as a sub-test	Graftable onto H2 or H3, without its own simulator
C2	rejected	Two adversarial reviews, dead on paper, no run

8.2 The corpus : a minuscule logic stratum

The catalogue enumerates 29 scenarios. The derived file `benchmarks/labels/derived\_index.json` counts 288 expected entries, of which 282 classified by the type field as detectable by signature and 6 as belonging to logic, that is 2.1 %. I recounted them entry by entry.

Two counts coexist and do not measure the same thing. The *mechanical* reading places every category except two in signature. The *interpretive* reading judges category by category whether a well-designed rule can conclude on its own ; by taking its signature categories on the index, one obtains 243 entries out of 288, that is 84.4 %, and the four readings indeed total 288. Both figures are legitimate but must never be cited one for the other.

Table 12 – Distribution of the 288 entries by category and interpretive reading.

Category	Count	Interpretive reading
`misconfiguration`	70	signature
`data\_exposure`	54	signature
`no\_authentication`	50	signature
`default\_credentials`	41	signature
`info\_disclosure`	16	signature
`code\_injection`	13	mixed
`insecure\_update`	12	mixed
`weak\_crypto`	10	mixed
`cve`	9	signature
`privilege\_escalation`	4	chaining
`broken\_access\_control`	4	logic
`missing\_header`	3	signature
`auth\_bypass`	2	logic
Total	288	243 signature, 35 mixed, 4 chaining, 6 logic

The logic stratum occupies only three scenarios, and only 55 entries are attached to a chaining of at least two hops. A global duel on this corpus would therefore measure above all the quality of the rules. This is exactly what H2 observed.

8.3 Genuine achievements

H2, measured on 22 October. The corpus contains eight dossiers, two per group, for 25 messages in total. The documented run executes three arms per dossier, that is 24 arms per repetition and 72 arms for the three repetitions. On the raw variants, the LLM arm obtains 0.5 (one dossier won out of two), the declared parser 0.0, and the third arm, which reads by convention without any key from the corpus, covers 1.0. The prediction holds in the literal sense, but the gain of the model is absorbed by the convention.

C2, rejected. The review concludes to a halt : the first candidate is self-contradictory, the second is poorly attached and has only non-conforming outcomes, and the count of families is not a measure of independence. No fixture, no run : this is a negative methodological result, publishable as a limitation.

8.4 Inference : repaired, and still unusable

The inference tool (`src/benchmark/inference.py`) carried a structural defect. It computed a two-sided p-value around zero, so every outcome returned the same figure. Executing the function confirmed it : an almost constant series, a highly dispersed series of the same mean, and a series entirely negative all produced a null p-value. A model performing strictly worse than the rules looked as certain as one performing better. The tool produced the appearance of an inference that it did not provide.

The defect was repaired on 30 October 2026. The test is now one-sided against the pre-registered threshold. An exact sign test is used up to seven families, a paired bootstrap beyond, both centred on the threshold rather than on zero. Below three families no p-value and no interval are emitted ; the observed mean remains only as a descriptive figure. A decision requires five families, because a lower bound of five agreeing families is the first that can reach the usual threshold at all.

Measured behaviour, five agreeing families : the difference is significant. Mixed results and a strictly losing model are both returned as not rejected. The threshold now enters the decision.

Repairing the computation did not make a conclusion possible. A second obstacle stands behind the first, and it is not a defect of the tool : the number of independent families available. The trial configuration includes the commit, the fingerprints and the budgets, so two distinct families form two distinct configurations ; each configuration contains only one family, and the inference requires several. Whatever the tool computes, no figure can be produced from families that do not exist.

8.5 A family built, then rejected in one hour

The report so far described the family as a concept. On 30 October 2026 a family was built far enough to be measured, and rejected. The record matters more than the rejection : it is the first time the project's own protocol produced a decision instead of a plan.

What was built. A register device serving twenty holding registers and as many input registers, a journal of raw reads and writes, a private oracle, and three arms : a declared rule set, a by-convention script, and a witness. The design passed six independent reviews, four revisions apart, and survived where earlier proposals had failed. Its addressing model was established against raw MBAP frames

rather than a client object, after three successive models had been declared verified while reading the wrong offset.

The trap. Three registers were prepared. One commands something and changes when written ; one is inert and does not change ; the third is a cache — it changes when written but commands nothing, and propagates its value into its neighbours. The by-convention arm reads the unit and the change : it classes the cache as a projection and scores two out of three. An arm reading the neighbours scores three out of three, and names the correct write in all three justifications.

Why it was rejected. Reading the neighbours is a rule, not an interpretation. Whether one places that rule inside the competent automation or outside it, the family measures nothing. Inside, the automation succeeds and the closure test rejects the family. Outside, the family measures an omission of the automation rather than a limit of interpretation. There is no third reading.

The lesson is stated in the record as it happened : a trap that a rule crosses is not a question of interpretation. The filter that would have caught it applies in minutes and costs nothing :

> If the rule that solves the case can be written in one line, a script will have it. Do not build the family.

What survives the rejection. Three findings remain, and they are what the next attempt inherits.

First, absence can be deduced. A register that does not change when written, after a declared test, is attested as non-commanding without any observation labelled “absent” in the journal. Labelling would have put the answer in the journal.

Second, a journal must be attributable. The probe exposed three defects of the same kind : a `null` value posing as an observation, reads that named no write, and a missing `before` phase that let a register inherit its neighbour’s history. The rule now written is that no observation may belong to two causes.

Third, the protocol works. Six reviews validated a design that one hour of observation rejected. The observation was cheaper than a campaign, and it produced a defensible negative result rather than a family that would have measured its own limit.

What the rejection does not prove. It does not refute H4. It shows that this mechanism, on this corpus, is crossed by a generic rule. A different mechanism could survive, and any future family on H4 must demonstrate before construction that no generic rule crosses it.

8.6 Campaign admission

The admission module combines pair conditions and campaign conditions. The condition for explicit adoption of the confirmation metrics is a parameter with a default value of false, never set to true by the summary nor by the command line. The module justifies this lock by a contract argument : the driver harnesses do not emit these counters. This is accurate as to the current state, but it is not a proof of unsatisfiability : the condition would become true again the day a harness publishes these counters. Today, it is therefore not satisfied.

A second defect, noted during drafting and then verified, deserves review. The budget-parity condition requires that the first value exists and that the two are equal ; the absence of a budget is therefore correctly refused, and this point complies with the announced rule. By contrast, the same module does not verify that missing budgets are distinguished from non-comparable budgets in the other conditions ; the review recommended this check without finding it again.

8.7 What is publishable today

The publishable element is the method, not a result of superiority. The repository supports no claim of comparative performance : there exists neither a demonstration that a model beats a competent script, nor a computable confidence interval, nor an independent family. What holds, by contrast, is consistent and verifiable :

— a falsifiable register of hypotheses, with contrast declared before the fixture, private oracle and measurement recomputed from the log ;
— a measurement lesson on H2 : a prediction can hold in the literal sense and demonstrate nothing, because an arm by convention absorbs the gain ;
— a clean negative result on C2, documented before any grey area ;
— an honest inventory of the holes : reduced logic stratum, unknown protocol absent, noise not encoded.

8.8 What is not demonstrated

Must not be written as established : the superiority of the LLM ; a significant gap, since the inference tool produces a p-value unrelated to the gap and publishing such a figure would be a fault ; generality, H2 covering only one schema, one question and one model ; the cost gain, the exploratory pilot counting only one pair without an interval ; the validity of the threshold, not applied by the gate ; and the integrity of the corpus, the official healthy control not being in the catalogue.

8.9 The other active studies

Plan V2 carries 44 records, 25 sets and several hundred conditional arms. The qualifications actually executed, SSH in October and MQTT a few days later, are recorded as qualifying the instrument, not an advantage. The three-repetition pilot yields twelve admissible pairs and computes no interval.

The historical study on the agent versus automation remains active and carries the S1 pair and the model-free programme. Its synthesis states it plainly : a real descriptive result, no general superiority established. This is the formulation to keep.

9 Conclusion

9.1 Three instruments, three levels of guarantee

The project assembles three instruments that do not offer the same guarantees, and the main source of confusion is to treat them as interchangeable.

Table 13 – What each instrument guarantees and does not guarantee.

Instrument	Guarantees	Does not guarantee
Execution boundary (`executor.py`)	Each tool call is logged, bounded and compliant with the scope	System-level isolation ; the guards are application-level
Sandbox (`sandbox.py`)	No listed Python event occurs ; the deterministic arms run outside the laboratory	Native isolation ; any model measurement
Calibration harness	Contract, separate oracle, evaluator that re-reads the log	Campaign admission ; exact parity of the arms

The first instrument makes the audit traceable. The second makes the harness portable. The third makes the comparison possible. None of them proves that a model does better than a script, which was the object of the programme.

9.2 The contradictions identified between sections

Confronting the five drafts brought out four discrepancies, all of them verified by reading the code before being retained here.

The threshold of the V0 check is not applied by the gate. The protocol framework asserts that the gate applies `X=0,9` ; the benchmark part verifies that the gate is computed on "all successes" and that the threshold check is independent. The two formulations coincide at three repetitions but diverge beyond that. The report retains the reading of the code.

The H2 corpus does not encode the described failure mode. The protocol states that, for a given record, abstention is the only correct answer ; reading the oracle shows that it expects an exposure on the first message and a non-exposure on the third. The by-convention arm moreover converges exactly on the oracle. The protocol therefore describes an intention that the corpus does not encode.

The blocking of inference is not the one commonly alleged. The admission condition is cited as the cause ; reading shows that it is satisfiable in principle, and that the real constraint is the absence of independent families. This shift matters : it changes the priority of the work.

The routing failure of 29 October is not recorded. It accounts for six hours of interruption and the rebuilding of the container, but no document in the repository describes it. This is a gap in the project's memory, not a disagreement between sections.

9.3 A common pattern : absorbed gain

Three episodes follow the same shape. In the H2 family, the model's gain over the declared parser is taken up by a script arm that reads by convention. In the sandbox, the same contrast is reproduced offline, without a model. In the rejection of C2, the targeted flaw turns out to be expressible from public inputs, and therefore closeable by a competent automation.

The pattern can be stated thus : *what distinguishes the model from the script, in the constructed families, is a limitation of the declaration rather than a capability of the model.* This is not a demonstration of inferiority ; it is the absence of a situation in which the difference appears. Yet such a situation is precisely the object of the programme.

9.4 What the project can assert today

Four assertions are defensible, each with its limitation.

An audit produces verifiable traces : each tool call is archived, referenced and bounded, and a lost piece of evidence interrupts the run rather than leaving a silent gap. The limitation is that these are application-level guards.

A harness can be executed outside the laboratory, without a broker or a model, under a safeguard whose blind spots are declared. The limitation is that it measures nothing : it instruments.

A third script arm absorbs a gain attributed to the model on the most documented family. The limitation is that this holds for eight fixtures and one property.

The corpus is mostly detectable by signature, which makes a null difference expected by construction. The limitation is that this concerns the current corpus, not the targeted situations.

9.5 What the project cannot assert

No superiority of the model is established. No confidence interval can be computed. No generalisation is possible from the existing measurements. The inference module, which produced the appearance of a measurement it did not provide, was repaired on 30 October 2026 : it now refuses to emit a figure it cannot defend. What remains is the obstacle behind it — the families that would give it something to compute do not exist.

9.6 The three decisions that govern what follows

A first decision concerns the independent families, now that the module can no longer produce an indefensible figure : to build them, applying the filter that one hour of observation established, or to publish a bounded methodological result and declare it as such. The filter matters more than the count : a family crossed by a generic rule measures nothing, and five of them measure nothing either.

A second decision concerns the opening condition : to write a competent reference automation, or to explicitly renounce the hypotheses that depend on it. This decision makes several families that are currently undecidable legible.

This is the decision that determines whether an inference can one day exist.

None of these decisions is technical in the narrow sense. All of them concern what one accepts not to be able to say.

9.7 A legible conclusion

The system is built, instrumented and documented. Its chain of evidence is sound : a separate oracle, an evaluator that recounts, a log that does not forgive a lost piece of evidence, a safeguard that declares its blind spots. The defects identified in this report were found by reading what the code does, not what it claims, and that is the best evidence that the verification discipline works.

The scientific question, however, remains open. The programme asked when a model does better than a script with identical tools ; the available data do not answer it, and the current arrangement would not allow an answer even with more machine time. This observation is not a failure of the work accomplished : it is the result of having looked at things squarely.

The most useful continuation is not to add a family, but to make falsifiable what today claims to be so.